

Security Posture Scorecard

Mock RFP Response — github.com/him-agni/security-posture-scorecard

Written as a practice exercise: technical answers a Solutions Engineer would give during a buyer evaluation, based on the architecture actually implemented in this project.

Q: Static-analysis tools have a reputation for false positives that erode developer trust. How does your platform avoid this?

A: Every finding is tagged with a confidence tier — 'verified' when the tool can prove it from source (like a repo-wide secret scan or a wide-open CORS configuration), 'detected' when it's a heuristic signal (a library imported but not necessarily used), and 'manual' when it's genuinely unverifiable from static analysis (encryption at rest, backups). Developers and security teams see what was proven vs. inferred vs. surfaced for human review, so the score is trustworthy and doesn't hide a false green.

Q: How is the security score actually calculated? We've been burned by black-box scoring before.

A: Scoring is fully transparent. Each check is weighted by severity — critical 40, high 25, medium 15, low 10. A pass earns full weight, a warn earns half, a fail earns zero. Manual-tier items are informational only and never penalize the score, since they can't be verified from source. Every report includes a 'why this score' breakdown showing each finding's weight and contribution.

Q: Our repositories vary — some are frontend libraries, some are full-stack apps. Will a frontend-only project get an unfair failing grade for missing backend controls like rate limiting?

A: No. The scanner detects when a layer is not applicable — a repo with no backend or database code has those layers marked N/A and excluded from scoring entirely, so a frontend library isn't penalized for missing controls it shouldn't have in the first place.

Q: How do you handle known-vulnerable dependencies?

A: The platform queries the OSV vulnerability database against the project's lockfile, so a known-vulnerable transitive dependency is flagged as a verified finding — not a heuristic. Lockfiles and node_modules are excluded from the source scan itself to avoid noise from third-party code.

Q: Our AppSec team has policies specific to our organization. Can we add our own checks?

A: Yes. Every check is a self-contained plugin module registered through a central registry, all sharing the same input contract. Adding a new check — an internal policy, a compliance-specific rule, a language-specific pattern — is an additive change without touching the scanner core.

Q: What guarantees do you have that scanning a repository won't leak our source code or hang on a large repo?

A: The scanner downloads a tarball from GitHub to a temporary directory, runs the checks against a single in-memory file tree, and always cleans up the temp directory — even on error, via a try/finally. There's a repo size cap and a whole-scan timeout, so a large or malformed repo can't hang the request or leak state between scans.